References

Error recovery and troubleshooting for bridges

Copy page

Identify bridge transfer failures, recover partial bridge transfers, and implement error handling for App Kit bridge transfers

Bridge transfers can encounter two error types. Hard errors stop execution. Soft errors let you recover and retry the bridge transfer. App Kit provides error handling that helps you respond in both cases.

Hard errors throw exceptions such as validation errors, configuration issues, and authentication problems. Soft errors occur during the bridge transfer but return enough transaction information for recovery. Examples include insufficient balance, network timeouts, and RPC connectivity issues.

This guide helps you identify failure points, recover partial bridge transfers, and implement error handling patterns.

Bridge transfer failures

This section explains how to identify where a bridge transfer failed and resume the bridge transfer manually.

Transaction steps overview

Each bridge transfer uses Circle's CCTP protocol provider, which breaks each transaction into various steps:

- `approve` : Allows the contract to spend USDC.
- `burn` : Burns USDC on the source blockchain and generates an attestation.
- `fetchAttestation` : Waits for Circle to sign the burn proof.

- `mint` : Mints USDC on the destination blockchain with the attestation.

Bridge result details

When a bridge transfer fails, App Kit returns a `BridgeResult` object showing which steps completed and which failed. This lets you resume the bridge transfer manually using the `CCTPv2BridgingProvider` , as shown in examples for **failed attestation fetch** and **failed mint**.

Focus on these `BridgeResult` properties during recovery:

- `result.state` - shows whether the bridge transfer succeeded or failed (`pending` , `success` , `error`)
- `result.steps` - each object contains:
  - `name` : the name of the step
  - `state` : the status of the step
  - `txHash` : the transaction hash if the step completed
  - `error` : an error message if the step failed

This example shows a returned `result` object for a transaction that failed when fetching an attestation:

Shell

```
result.state: 'error'
result.steps: [
  { name: 'approve', state: 'success', txHash: '0x123...' },
  { name: 'burn', state: 'success', txHash: '0x456...' },
  { name: 'fetchAttestation', state: 'error', error: 'Network timeout' },
]
```

> **Note**: The `result.source` and `result.destination` from `BridgeResult` only contain address and blockchain properties. To use provider methods for recovery, you need to reconstruct full wallet contexts with adapter and chain properties.

Step analysis

This example shows how to check for completed steps and use a helper function to find specific steps:

TypeScript

```typescript
// Start a bridge transfer that might fail
const result = await kit.bridge({
  from: { adapter: sourceAdapter, chain: "Ethereum_Sepolia" },
  to: { adapter: destAdapter, chain: "Arc_Testnet" },
  amount: "1.00",
});

// Check which steps completed successfully
console.log("Bridge transfer state:", result.state);
console.log("Steps:", result.steps);

// Helper function to find specific steps
const getStep = (stepName: string) =>
  result.steps.find((step) => step.name === stepName);
const approveStep = getStep("approve");
const burnStep = getStep("burn");
const attestationStep = getStep("fetchAttestation");
const mintStep = getStep("mint");
```

Recovery scenarios

This section describes how you can implement recovery patterns.

Retry a failed bridge transfer

If a bridge transfer fails, you can retry it with the `retry` method. Pass the failed `BridgeResult` and the `to` and `from` adapters.

This example shows how the retry method works:

TypeScript

```
const result = await kit.bridge({
  from: { adapter, chain: "Ethereum_Sepolia" },
  to: { adapter, chain: "Arc_Testnet" },
  amount: "1.00",
});

if (result.state === "error") {
  const retryResult = await kit.retry(result, {
    from: adapter,
    to: adapter,
  });
  console.log(inspect(retryResult, false, null, true));
} else {
  console.log(inspect(result, false, null, true));
}
```

Retry after a failed mint step

This example forces the mint step to fail and then retries with a valid `to` adapter:

TypeScript

```typescript
import { createViemAdapterFromPrivateKey } from "@circle-fin/adapter-viem-v2";
import { AppKit } from "@circle-fin/app-kit";
import type { BridgeResult } from "@circle-fin/app-kit";
import { inspect } from "node:util";

const adapter = createViemAdapterFromPrivateKey({
  privateKey: process.env.PRIVATE_KEY as string,
});

// This is a fake adapter to force an error at the mint step
const fakeAdapter = createViemAdapterFromPrivateKey({
  privateKey: ("0x" + "1".repeat(64)) as string,
});

const findErrorStep = (result: BridgeResult) => {
  if (result.state === "error") {
    return result.steps.find((step) => step.state === "error");
  }
  return null;
};

const kit = new AppKit();

const result = await kit.bridge({
  from: { adapter, chain: "Ethereum_Sepolia" },
  to: { adapter: fakeAdapter, chain: "Arc_Testnet" },
  amount: "1.00",
});

console.log("INITIAL RESULT", inspect(result, false, null, true));

if (result.state === "error") {
  const errorStep = findErrorStep(result);
  if (
    errorStep &&
    errorStep.errorMessage?.includes("gas required exceeds allowance") // This
  ) {
    const retryResult = await kit.retry(result, {
      from: adapter,
      to: adapter, // To succeed this time we're using the correct adapter
    });
```

```
  console.log("RETRY RESULT", inspect(retryResult, false, null, true));
}
```

Common issues

This section lists common issues and solutions.

Insufficient balance

Ensure you have enough USDC in your wallet before a bridge transfer to avoid an insufficient balance error.

This example checks your wallet balance:

TypeScript

```
import { createViemAdapterFromPrivateKey } from "@circle-fin/adapter-viem-v2";
import { formatUnits } from "viem";

const adapter = createViemAdapterFromPrivateKey({
  privateKey: process.env.PRIVATE_KEY as string,
});

const balanceAction = await adapter.prepareAction(
  "usdc.balanceOf",
  {},
  { chain: "Arc_Testnet" },
);
const balance = await balanceAction.execute();
console.log(`USDC balance: ${formatUnits(BigInt(balance), 6)}`);
```

Transaction stuck or failed

If a transaction is stuck or failed, check the transaction on a block explorer with the returned `txHash`. For Solana bridge transfers, use Solana Explorer or SolScan.

If the transaction failed during the bridge transfer, check the returned `result.steps` to see which transaction steps completed.

Best practices

Follow these practices for prevention, recovery, and monitoring to improve reliability.

Prevention

- Test your integration on testnets before deploying on mainnet.
- Monitor gas prices and adjust during network congestion.
- Use dedicated RPC providers such as Alchemy or QuickNode.
- Implement multiple RPC fallbacks.
- Wrap all bridge transfers in try-catch including adapter setup and bridge calls.

Recovery

- Always save the bridge transfer state for recovery scenarios.
- Verify which steps completed before attempting recovery.
- Use appropriate timeouts and give network operations enough time to complete.
- Implement exponential backoff and use increasing delays for retry logic.

Monitoring and debugging

- Use block explorers to verify transaction status.
- Save intermediate results and persist bridge transfer state for recovery scenarios.

Was this page helpful? Yes No

How bridge fees work Swap overview

Network status
Blog
Community
Arc homepage
Terms of service
Privacy policy
Cookie policy

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.